

UNIVERSITY OF WAH(UOW)

**DEPARTMENT OF COMPUTER
SCIENCE**



COURSE:

Information & Security lab - 251L

SUBMITTED TO:

Ms. Wasifa Kanwal

SUBMITTED BY:

Muhammad Talha

Section

CS.3ndB

Registration NO:

UW-24-CS-BS-101

Lab Task # 04

Task # 01

```
def caesar_encrypt(text, key):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        else:
            result += ch
    return result

def caesar_decrypt(cipher, key):
    return caesar_encrypt(cipher, -key)

encrpyted = caesar_encrypt("hello world",3)
print(encrpyted)
print("Decrypted: ",caesar_decrypt(encrpyted,3))
```

Output

```
KHOOR ZRUOG
Decrypted: HELLO WORLD
```

Code

```
def caesar_encrypt(text, key):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        else:
            result += ch
    return result

def caesar_decrypt(cipher, key):
    return caesar_encrypt(cipher, -key)

encrpyted = caesar_encrypt("HELLO WORLD",3)
print(encrpyted)
print("Decrypted: ",caesar_decrypt(encrpyted,3))
```

Task # 02

```
import string, random
# step 1
alphabet = list(string.ascii_uppercase)
print(alphabet)
key = alphabet.copy()
random.shuffle(key)
print(key)
# step 2
encrypt_map = {}
decrypt_map = {}
for i in range(len(alphabet)):
    encrypt_map[alphabet[i]] = key[i]
    decrypt_map[key[i]] = alphabet[i]
message = "DEAF"
cipher = ""
for letter in message:
    if letter.isalpha():
        cipher += encrypt_map[letter]
    else:
        cipher += letter
plain = ""

for letter in cipher:
    if letter.isalpha():
        plain += decrypt_map[letter]
    else:
        plain += letter

print("PlainText: ",message)
print("Encrypted Text: ",cipher)
print("Decrypted Text: ",plain)
```

Output

```
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
['S', 'I', 'M', 'W', 'A', 'N', 'X', 'U', 'Y', 'V', 'K', 'E', 'G', 'Q', 'O', 'Z', 'F', 'T', 'D', 'H', 'J', 'C', 'R', 'P', 'L', 'B']
PlainText: DEAF
Encrypted Text: WASN
Decrypted Text: DEAF
PS C:\Users\pc\Documents\Python Files> []
```

Code

```
import string, random
# step 1
alphabet = list(string.ascii_uppercase)
print(alphabet)
key = alphabet.copy()
random.shuffle(key)
print(key)
# step 2
encrypt_map = {}
```

```

dencrypt_map = {}
for i in range(len(alphabet)):
    encrypt_map[alphabet[i]] = key[i]
    dencrypt_map[key[i]] = alphabet[i]
message = "DEAF"
cipher = ""
for letter in message:
    if letter.isalpha():
        cipher += encrypt_map[letter]
    else:
        cipher += letter
plain = ""

for letter in cipher:
    if letter.isalpha():
        plain += dencrypt_map[letter]
    else:
        plain += letter

print("PlainText: ",message)
print("Encrypted Text: ",cipher)
print("Decrypted Text: ",plain)

```

Task # 03

```

def vigenere_encrypt(text, key):
    cipher = ""
    key = key.upper()

    for i, ch in enumerate(text.upper()):
        if ch.isalpha():
            shift = ord(key[i % len(key)]) - 65
            text += chr((ord(ch) - 65 + shift)%26 + 65)
        else:
            text += ch
    return text

def vigenere_decrypt(text, key):
    cipher = ""
    key = key.upper()

    for i, ch in enumerate(text.upper()):
        if ch.isalpha():
            shift = ord(key[i % len(key)]) - 65
            text += chr((ord(ch) - 65 - shift)%26 + 65)
        else:
            text += ch
    return text

plain_txt = "ATTACKATDAWN"
keyword = "LEMON"

```

```
keyword = "LEMON"
cipher = vigenere_encrypt(plain_txt,keyword)
print("Encrypted: ",cipher)
decryption = vigenere_dencrypt(cipher, keyword)
print("Decrypted: ",decryption)
```

Output

```
Encrypted:  ATTACKATDAWNLXFOPVEFRNHR
Decrypted:  ATTACKATDAWNLXFOPVEFRNHRPPHPZWHPNLJZJSDLJQSGJVD
```

Code

```
def vigenere_encrypt(text, key):
    cipher = ""
    key = key.upper()

    for i, ch in enumerate(text.upper()):
        if ch.isalpha():
            shift = ord(key[i % len(key)]) - 65
            text += chr((ord(ch) - 65 + shift)%26 + 65)
        else:
            text += ch
    return text

def vigenere_dencrypt(text, key):
    cipher = ""
    key = key.upper()

    for i, ch in enumerate(text.upper()):
        if ch.isalpha():
            shift = ord(key[i % len(key)]) - 65
            text += chr((ord(ch) - 65 - shift)%26 + 65)
        else:
            text += ch
    return text

plain_txt = "ATTACKATDAWN"
keyword = "LEMON"
cipher = vigenere_encrypt(plain_txt,keyword)
print("Encrypted: ",cipher)
decryption = vigenere_dencrypt(cipher, keyword)
print("Decrypted: ",decryption)
```

Task # 04

```
def encrypt(text, key):
    result = ""
    for char in text:
        if char.isalpha():
            # Preserve case (upper/lower)
            shift = 65 if char.isupper() else 97
            result += chr((ord(char) - shift + key) % 26 + shift)
        else:
            result += char
    return result

def decrypt(ciphertext, key):
    result = ""
    for char in ciphertext:
        if char.isalpha():
            shift = 65 if char.isupper() else 97
            result += chr((ord(char) - shift - key) % 26 + shift)
        else:
            result += char
    return result

plaintext = input("Enter your message: ")
key = int(input("Enter numeric key: "))
ciphertext = encrypt(plaintext, key)
print("\nEncrypted message:", ciphertext)
decrypted_text = decrypt(ciphertext, key)
print("Decrypted message:", decrypted_text)
```

Output

```
Enter your message: Talha is doing Security Lab
Enter numeric key: 5

Encrypted message: Yfqmf nx itnsl Xjhzwnyd Qfg
Decrypted message: Talha is doing Security Lab
PS C:\Users\pc\Documents\Python Files>
```

Code

```
def encrypt(text, key):
    result = ""
    for char in text:
        if char.isalpha():
            # Preserve case (upper/lower)
            shift = 65 if char.isupper() else 97
            result += chr((ord(char) - shift + key) % 26 + shift)
        else:
            result += char
    return result

def decrypt(ciphertext, key):
    result = ""
    for char in ciphertext:
        if char.isalpha():
            shift = 65 if char.isupper() else 97
```

```

        result += chr((ord(char) - shift - key) % 26 + shift)
    else:
        result += char
    return result
plaintext = input("Enter your message: ")
key = int(input("Enter numeric key: "))
ciphertext = encrypt(plaintext, key)
print("\nEncrypted message:", ciphertext)
decrypted_text = decrypt(ciphertext, key)
print("Decrypted message:", decrypted_text)

```

Task # 05

```

import string, random

alphabet = list(string.ascii_uppercase)
key = alphabet.copy()
random.shuffle(key)
encrypt_map = {alphabet[i]: key[i] for i in range(26)}
decrypt_map = {key[i]: alphabet[i] for i in range(26)}
message = input("Enter your message: ")
cipher = ""
for letter in message:
    if letter.isalpha():
        if letter.isupper():
            cipher += encrypt_map[letter]
        else:
            cipher += encrypt_map[letter.upper()].lower()
    else:
        cipher += letter
plain = ""
for letter in cipher:
    if letter.isalpha():
        if letter.isupper():
            plain += decrypt_map[letter]
        else:
            plain += decrypt_map[letter.upper()].lower()
    else:
        plain += letter
print("\nEncryption Key Map:")
print(encrypt_map)
print("\nPlain Text: ", message)
print("Encrypted Text:", cipher)
print("Decrypted Text:", plain)

```

Output

```

Enter your message: Muhammad Talha

Encryption Key Map:
{'A': 'S', 'B': 'R', 'C': 'B', 'D': 'Q', 'E': 'U', 'F': 'N', 'G': 'X', 'H': 'K', 'I': 'H', 'J': 'C', 'K': 'L', 'L': 'I', 'M': 'P', 'N': 'E', 'O': 'O', 'P': 'D', 'Q': 'Z', 'R': 'J', 'S': 'W', 'T': 'A', 'U': 'Y', 'V': 'G', 'W': 'M', 'X': 'T', 'Y': 'F', 'Z': 'V'}

Plain Text: Muhammad Talha
Encrypted Text: Pyksppsq Asiks
Decrypted Text: Muhammad Talha

```

Code

```
import string, random

alphabet = list(string.ascii_uppercase)
key = alphabet.copy()
random.shuffle(key)
encrypt_map = {alphabet[i]: key[i] for i in range(26)}
decrypt_map = {key[i]: alphabet[i] for i in range(26)}
message = input("Enter your message: ")
cipher = ""
for letter in message:
    if letter.isalpha():
        if letter.isupper():
            cipher += encrypt_map[letter]
        else:
            cipher += encrypt_map[letter.upper()].lower()
    else:
        cipher += letter
plain = ""
for letter in cipher:
    if letter.isalpha():
        if letter.isupper():
            plain += decrypt_map[letter]
        else:
            plain += decrypt_map[letter.upper()].lower()
    else:
        plain += letter
print("\nEncryption Key Map:")
print(encrypt_map)
print("\nPlain Text: ", message)
print("Encrypted Text:", cipher)
print("Decrypted Text:", plain)
```